Look Before You Encode: Foveated Tile Streaming for Resolution-Independent VLM Inference

Anonymous CVPR submission

Abstract

High-resolution vision-language models (VLMs) split an image into many tiles, and their visual tokens dominate the memory of inference: an 8K image yields 48K tokens, and a 48 GB GPU holds only three such requests. Visual token pruning cannot bound this cost, because it discards tokens only after they have been computed. We propose FoveaTile, a training-free method that decides which regions to encode before *encoding them. A single partial pass of the VLM over low-resolution coverage tiles ranks the high-resolution detail tiles by their relevance to the question, and only the selected tiles are encoded and streamed into the KV cache, so memory and compute are set by the token budget instead of the input resolution. On InternVL3.5 and six high-resolution benchmarks, FoveaTile retains 95.0% of the uncompressed accuracy with 5.0× less memory and is the most accurate method under every memory limit. Its peak memory stays at 0.41 GB from 7 to 188 tiles, so one GPU serves at least 3.3× more requests than in-LLM pruning and 41× more than the uncompressed model.*

1. Introduction

Memory is the main constraint in serving large language models. The KV cache grows with every token in the context, and it, not compute, limits how many requests a GPU can hold at once [6], which has motivated a line of work on compressing it [25, 32]. Vision-language models (VLMs) make this problem much worse. To read small text and recognize small objects, current VLMs split a high-resolution image into many tiles at near-native resolution [4, 10, 21], and each tile becomes hundreds of visual tokens. Kept at the native tile resolution of InternVL3.5-8B [21], an 8K image needs 188 tiles, *i.e.* 48K visual tokens. Its prefill needs 12.9 GB of GPU memory on top of the model weights and 17 s to the first token, so a 48 GB GPU can hold only three such requests. Visual tokens, not text, now dominate the memory of a VLM, and their number grows linearly with the image area.

Visual token pruning reduces the number of tokens that reach the language model, but it acts too late. VisionZip [28] prunes after the vision encoder, so every tile is still encoded. FastV [3], SparseVLM [30], and IF-Prune [18] prune inside the language model, so all tokens still pass through its first layers, and the peak memory of prefill is still set by the full image. At 188 tiles these methods need more than 8 GB, although they keep only 1,536 tokens; IF-Prune, which adds a guide model, needs more memory than the uncompressed model already at the default resolution. The obvious alternative, downscaling the image before encoding, loses exactly the details that high resolution was meant to provide: on V* Bench [24], 80% of the target objects become smaller than a single ViT patch in the global thumbnail. Local crops alone, on the other hand, lose the layout and context of the whole image. Yet the answer usually lies in a small part of the image: on V* Bench, the target objects cover a median of 0.08% of the image and fall into a single one of 24 tiles, while token pruning encodes all of them.

The question is therefore not how many visual tokens to keep, but which regions to compute at all. To bound memory without losing these details, this decision must be made *before* the regions are encoded (Fig. 2). This is feasible: low-resolution attention alone places the target within four of a median of 24 tiles on 67.0% of the V* Bench questions, against 14.8% for random selection (Sec. 3.2). We propose FoveaTile, a training-free method that follows the principle *look before you encode*. Like foveated vision [14, 23], it combines a coarse view of the whole scene with a few high-resolution glimpses. FoveaTile spends half of the token budget on low-resolution *coverage tiles* of the whole image. A single partial forward pass of the VLM over these tiles and the question yields a question-aware prior over the high-resolution *detail tiles*; only the tiles at the selection boundary are re-scored with question attention. The selected detail tiles are then encoded and streamed into the KV cache, and all other tiles never reach the vision encoder. As a result, the vision encoder, the language model, and the KV cache all process a number of tokens set by the budget, independently of the input resolution, and no model other than the VLM itself is needed.

Our contributions are as follows.

- We identify that existing visual token pruning cannot

bound the memory of high-resolution VLMs, because it removes tokens only after they have been computed, and we propose to select image regions before encoding instead. We show on V* Bench that low-resolution attention can filter regions by question, and that re-scoring a few tiles at high resolution recovers most of the targets it would miss.

- We present FoveaTile, which ranks tiles with the VLM's own attention from a low-resolution view, re-scores only the uncertain tiles, and builds the KV cache by streaming. Its vision compute, prefill, and KV cache are $\mathcal{O}(K)$ in the budget K and independent of the number of tiles.
- On InternVL3.5 and six high-resolution benchmarks, FoveaTile keeps 95.0% of the uncompressed accuracy with 5.0× less memory, and it is the most accurate method under every memory limit. From 7 to 188 tiles its peak memory stays at 0.41 GB, 20× less than pruning inside the language model, so one GPU serves 3.4× and 3.3× more requests than FastV and IF-Prune and 41× more than the uncompressed model (Fig. 5); beyond about 35 tiles it is also the fastest method. The memory advantage holds from 4B to 14B parameters without retuning.

2. Related Work

High-resolution VLMs. Most open VLMs reach near-native resolution by dynamic tiling: the image is split into fixed-size crops that are encoded independently and appended with a downscaled global view [4, 7, 9, 10, 21, 27]; others encode the image at its native resolution directly [19]. Either way, the number of visual tokens, and with it the cost of the vision encoder, the prefill, and the KV cache, grows linearly with the image area.

Visual token compression. Training-free methods reduce visual tokens either after the vision encoder, using its attention or token similarity [2, 16, 28], or inside the language model, guided by text-to-image attention [3, 26, 30] or information flow [18]. For tiled high-resolution inputs, the global view can also distribute the token budget over the crops [1, 11]. All of them select among tokens that have already been computed (Sec. 3.1). FoveaTile selects before the vision encoder and keeps whole tiles rather than scattered tokens, which preserves the spatial layout of text and small objects.

KV cache compression. For language models, the KV cache is compressed by evicting tokens with low accumulated attention [32] or by keeping only attention sinks and a recent window [25], and paged memory management packs the caches of many requests on one GPU [6]. Eviction acts after the cache has been built, so it reduces the memory held during decoding but not the peak of prefilling a long visual sequence. FoveaTile is complementary: it never builds the full cache, and since its cache has a fixed size per request, it fits paged serving directly.

Zooming and visual search. Downscaling the whole image is not an alternative, since small targets fall below the resolution of a single patch. Zoom-in methods therefore locate the relevant region and look at it again at higher resolution, using the attention of the MLLM [29], a trained search model [24], tree search [8, 17], or an external retriever [22]. They aim to *add* detail on top of the full input and run the model several times per question, so they cost more than the uncompressed model. FoveaTile uses the same signal, *i.e.* where the VLM looks, to decide in a single partial pass over a low-resolution view which detail tiles are worth encoding at all.

3. Background and Motivation

High-resolution VLMs pay for the whole image, although the answer usually lies in a small part of it. We first show where this cost comes from and why token pruning does not remove it, and then ask whether a low-resolution view is enough to find the regions that matter.

3.1. Where the cost of high resolution comes from

We build on dynamic-resolution VLMs [4], using InternVL3.5 [21] as the running example. An image of size $W \times H$ is resized to a grid of $c \times r$ tiles of 448×448 pixels, where (c, r) is the grid whose aspect ratio is closest to W/H subject to $N = cr \leq N_{\max}$; a global thumbnail is appended when $N > 1$. Each tile is encoded independently by the vision encoder, and pixel shuffle followed by an MLP projector turns it into $n = 256$ visual tokens. The language model then prefills the sequence

$$[\,\mathbf{x}_b;\ \mathbf{V}_1;\ \ldots;\ \mathbf{V}_{N+1};\ \mathbf{x}_a\,], \tag{1}$$

where $\mathbf{x}_b$ is the text before the image (system prompt and user prefix), $\mathbf{V}_t \in \mathbb{R}^{n \times d}$ are the tokens of tile t, and $\mathbf{x}_a$ is the question, the answer instruction, and the assistant prefix.

Every stage of this pipeline grows with N: vision-encoder compute is $\mathcal{O}(N)$, the prefill of the language model is $\mathcal{O}(N)$ in its linear layers and $\mathcal{O}(N^2)$ in attention, and the KV cache is $\mathcal{O}(N)$. With the cap $N_{\max} = 47$ used in our experiments, a 4K image already yields 24 to 45 tiles (6K to 12K visual tokens); without a cap, an 8K image yields 188 tiles (48K visual tokens). Token-pruning methods remove tokens *after* they are computed. VisionZip [28] prunes after the vision encoder, so the encoder still sees all N tiles. FastV [3], SparseVLM [30], and IF-Prune [18] prune inside the language model, so the encoder and the first layers still

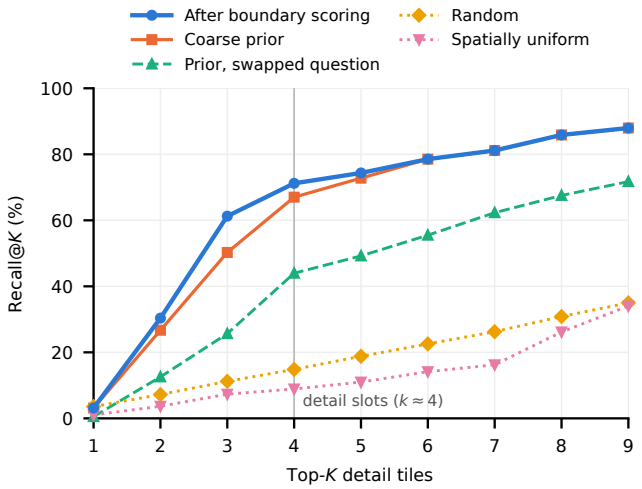


Figure 1. **Reliability of low-resolution attention** on V* Bench ($B = 1536$). Recall@K is the fraction of questions whose target is covered by the top K detail tiles. Swapped question: the prior computed with the question of another image of the same category.

B	K	Random	Uniform	Coarse prior	After scoring
1024	2	7.2	3.7	27.7	**69.1**
1536	4	14.8	8.9	67.0	**71.2**
2048	4	14.8	8.9	59.7	**72.8**
3072	6	22.6	14.1	77.5	**80.1**

Table 1. **Recall@K (%) on V* Bench** at about the number of detail tiles each budget keeps, with K fixed so that all columns are comparable. Random and uniform selection do not depend on the budget.

B	Window	Prior	Recov.	Lost	Final	Rate (rand.)
1024	2 of 5	53	+79	−0	**69.1%**	89.8% (33.8%)
1536	2 of 4	126	+17	−7	**71.2%**	77.3% (39.4%)
2048	2 of 4	115	+27	−2	**73.3%**	79.4% (33.8%)
3072	1 of 3	149	+9	−4	**80.6%**	64.3% (31.0%)

Table 2. **From the prior to the final selection** on the 191 questions of V* Bench. Prior: questions on which the prior alone covers the target with the tiles of the budget. Recov.: the target enters the window, the prior alone would miss it, and boundary scoring keeps it. Lost: the reverse. Final = (prior + recovered − lost) / 191. Rate: the recovery rate, computed over the questions whose target enters the window and that the prior alone would miss (88, 22, 34, 14 questions); in parentheses, the expected rate when the window tiles are kept at random.

process all nN tokens, and the peak memory of prefill is still set by N.

Cost therefore follows the input, not what is kept. On HR-Bench 8K with 188 tiles and $B = 1536$, FastV, SparseVLM, and IF-Prune keep only 3.2% of the 48K visual tokens, yet they need more than 8 GB of memory and 3.2 to 7.0 s to the first token. VisionZip keeps its memory flat but still encodes all 188 tiles (Tab. 6). The limit of these methods is thus not which tokens they keep, but that every tile is computed first.

3.2. How reliable is low-resolution attention?

Encoding only a few tiles means choosing them from a view in which the target may not even be resolved: on V* Bench [24], 80% of the target objects are smaller than one ViT patch in the thumbnail. We measure how reliable this choice is on V* Bench, the only one of our benchmarks with target boxes. A question counts as a hit if at least half of the box of each target lies in the chosen tiles. By this measure little has to be found: of a median of 24 detail tiles, one tile covers the target on 78.5% of the questions, and two tiles on all of them.

We examine two signals, which Sec. 4 defines in detail. The *coarse prior* ranks the detail tiles by the attention that the question pays to a low-resolution view of the image (Sec. 4.2). *Boundary scoring* then re-scores a small window of tiles near the selection cut-off with question attention on their high-resolution tokens (Sec. 4.3). We analyze the frozen method (Sec. 4.5), and no parameter was chosen from this analysis.

The prior can filter. With four of a median of 24 tiles, the coarse prior covers the target on 67.0% of the questions, against 14.8% for random and 8.9% for spatially uniform selection (Fig. 1). The ranking also follows the question: replacing the question with that of another image lowers recall to 44.0%, a difference of 23.0 points (95% CI [15.7, 30.9]). Regions can therefore be filtered by their relevance to the question before any detail tile is encoded.

The decision needs high resolution. The prior alone cannot make the final choice: its first-ranked tile covers the target on only 3.1% of the questions, so the top of the ranking cannot be taken at face value. Scoring only a few tiles at high resolution improves the choice at the cut-off. At the same K, boundary scoring raises recall at every budget, from 27.7% to 69.1% at $B = 1024$ and from 67.0% to 71.2% at the default $B = 1536$ (Tab. 1). Of the targets that enter the window and that the prior alone would miss, 64 to 90% are recovered, 2.0 to 2.7 times the rate of random selection (Tab. 2). The coverage view stays in the context in all cases, so this recall is not the final accuracy, which Sec. 5.2 reports.

3.3. Motivation

These observations explain three design choices of FoveaTile:

- Cost arises when tiles are computed, so the decision is

made before the vision encoder, and the KV cache is built by streaming.
- Low-resolution attention finds candidates for the question, so the VLM's own attention over a low-resolution view can filter the tiles, without any other model.
- The final choice needs high resolution, but only near the cut-off, so only a small boundary window is encoded and scored.

Problem setting. Given a budget of B visual tokens, *i.e.* $K = \lfloor B/n \rfloor$ tiles, we want to choose *before encoding* which tiles to keep, such that the vision encoder, the language model, and the KV cache each process $\mathcal{O}(K)$ tokens independently of N. The method is training-free and uses no model other than the VLM itself.

4. Method

FoveaTile decides *which* image regions to encode before any of them reaches the vision encoder. Given a token budget, it keeps a low-resolution view of the whole image, uses the VLM itself to rank the high-resolution tiles by their relevance to the question, and streams only the selected tiles into the KV cache. Tiles that are not selected are never encoded. As a result, the vision encoder, the language model, and the KV cache operate on a number of tokens set by the budget, not by the input resolution. Fig. 2 gives an overview; Algorithm 1 lists the entire procedure.

4.1. Coverage tiles and detail tiles

FoveaTile represents the image at two scales and splits the K tile slots between them.

Coverage tiles. The whole image is resized to a $c_c \times r_c$ grid of at most m tiles, where $m = \max(1, \lfloor K/2 \rfloor)$. The official tiling optimizes only the aspect-ratio match and often leaves part of such a budget unused. We instead pick the grid with the *most* tiles, $c_c r_c \le m$, among those whose aspect-ratio distortion is at most $1.75\times$, so that the coverage tiles use their share of the budget. The coverage tiles are always kept. They give the model the global layout and context of the image, and they are the input to the prior (Sec. 4.2).

Detail tiles. The detail tiles are the official $c \times r$ grid with $N \le N_{\max} = 47$ tiles, *i.e.* close to native resolution. FoveaTile keeps $k = K - c_c r_c$ of them, so slots that the coverage grid cannot fill go to detail tiles.

With $m = 1$ the coverage tile reduces to the official thumbnail. Splitting the budget evenly between the two scales is a calibrated choice (Sec. 4.5): document-style questions benefit from more coverage, while questions about small, local details need detail tiles.

4.2. Question-aware prior from coverage tiles

To rank the detail tiles without encoding them, we ask the VLM where it looks when it reads the question against the low-resolution view. This builds on the observation that MLLMs attend to the relevant region even when they cannot resolve its details [29], which we quantified in Sec. 3.2. Zoom-in methods exploit this by searching over crops with repeated forward passes or an external retriever [8, 17, 22, 24]; we instead compute a single prior from the coverage tiles, before any detail tile is encoded. We run the language model on $[\mathbf{x}_b;\ \mathbf{V}^c;\ \mathbf{x}_a]$, where $\mathbf{V}^c \in \mathbb{R}^{nm \times d}$ are the coverage tokens, and stop after layer l_{hi}. Let $\mathbf{A}^{(l)}$ be the head-averaged attention of layer l. The saliency of coverage patch j is the attention it receives from all text positions after the image, averaged over a band of middle-to-late layers $\mathcal{L}_p = \{l_{\text{lo}}, \dots, l_{\text{hi}}\}$, where visual information is transferred to the question tokens [33]:

$$s_j = \frac{1}{|\mathcal{L}_p|} \sum_{l \in \mathcal{L}_p} \frac{1}{|\mathbf{x}_a|} \sum_{i \in \mathbf{x}_a} \mathbf{A}^{(l)}_{i,j}. \tag{2}$$

After min-max normalization, each coverage patch is assigned to the detail tile that contains its center in normalized image coordinates, and the prior of detail tile t is the mean saliency of its patches, $\pi_t = \frac{1}{|P_t|} \sum_{j \in P_t} s_j$, again min-max normalized.

This pass costs a single forward over $|\mathbf{x}_b| + nm + |\mathbf{x}_a|$ tokens through l_{hi} of the L layers, which is independent of N. We compute only the attention rows of $\mathbf{x}_a$ in Eq. (2) and keep the fused attention kernel for the forward pass, so the full attention matrix is never materialized. The pass also produces the KV cache of the prefix $[\mathbf{x}_b;\ \mathbf{V}^c]$, which the next stage reuses.

4.3. Boundary scoring with question attention

The prior is computed at low resolution, so it can bring the target close to the top of the ranking but cannot make the final choice on its own (Sec. 3.2). FoveaTile therefore spends additional computation only on the tiles at the selection boundary (Fig. 3), similar to retrieve-then-rerank pipelines that apply an expensive scorer only to the candidates of a cheap first stage [15]. Let the detail tiles be sorted by π_t, let k be the number of detail slots, and let S be the size of the scoring window. With $a = \min(k, \lfloor S/2 \rfloor)$:

1. the top $k - a$ tiles are *kept directly* (encoded, not scored);
2. the next S tiles form the *boundary window*; they are encoded and scored, and the a highest-scoring ones are kept;
3. all remaining tiles are *discarded* and never reach the vision encoder.

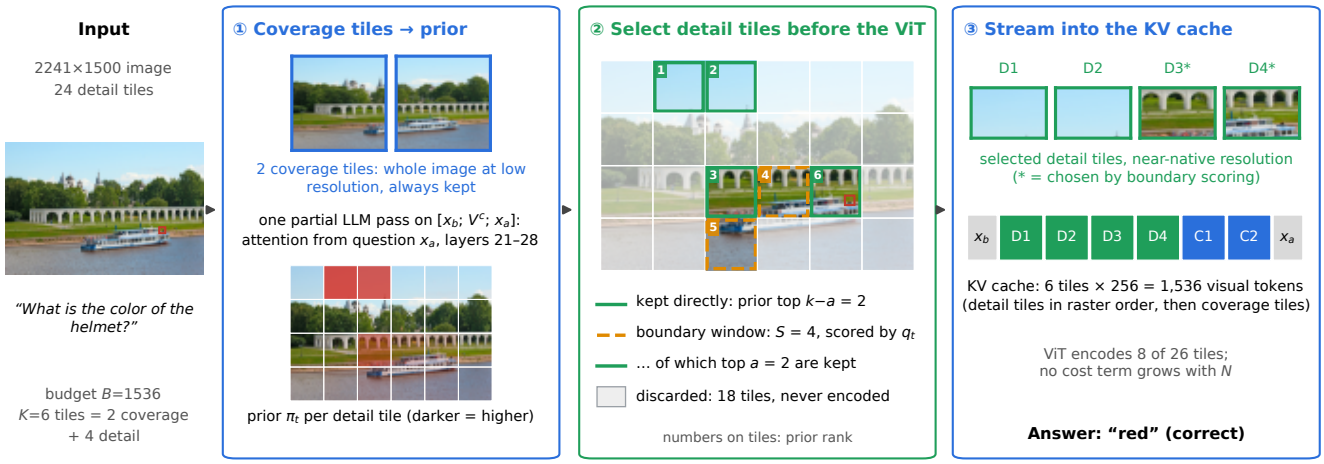


Figure 2. **Overview of FoveaTile.** The image is represented at two scales: m coverage tiles that show the whole image at low resolution, and a grid of N detail tiles at near-native resolution. A single partial forward pass over the coverage tiles yields a question-aware prior over the detail tiles. The top-ranked tiles are kept directly, the tiles at the selection boundary are re-scored with question attention, and all other tiles are discarded *before* the vision encoder. The selected tiles are streamed into the KV cache, so compute and memory are bounded by the budget K rather than by N. Example from V* Bench [24] with $B = 1536$ ($K = 6$); the 3:2 image fits a 2×1 coverage grid, so the spare slot goes to the detail tiles. The prior ranks the target tile only sixth, and boundary scoring recovers it.

The score of a boundary tile t is its question attention at layer l_q, measured in the context of the coverage tiles:

$$q_t = \frac{1}{|\mathbf{x}_a|} \sum_{i \in \mathbf{x}_a} \sum_{j \in \mathbf{V}_t} \mathbf{A}_{i,j}^{(l_q)} \quad \text{on} \quad [\,\mathbf{x}_b;\ \mathbf{V}^c;\ \mathbf{V}_t;\ \mathbf{x}_a\,]. \tag{3}$$

Because the prefix $[\,\mathbf{x}_b;\ \mathbf{V}^c\,]$ is shared by all candidates, its keys and values for layers $1, \ldots, l_q$ are taken from the prior pass, as in prompt caching [5]. Each candidate therefore costs one pass of $n + |\mathbf{x}_a|$ tokens through l_q layers, and this is mathematically identical to a full forward over the sequence in Eq. (3). Since q_t depends only on tile t and the question, the top-a selection can be made on the fly with a size-a min-heap while the window is streamed, without holding all candidates in memory. Setting $S = 0$ reduces FoveaTile to prior-only selection.

4.4. Streaming KV construction

FoveaTile builds the KV cache incrementally. It first prefills $\mathbf{x}_b$, then encodes the coverage tiles and runs the prior. The candidate detail tiles, *i.e.* the directly kept tiles and the boundary window, are encoded in raster order in micro-batches of four tiles; boundary tiles are scored as they arrive and enter or leave the size-a pool. Once selection is final, the selected detail tiles are written to the KV cache in raster order, followed by the m coverage tiles. This mirrors the order seen in training, where grid tiles precede the global thumbnail. The visual tokens are prefilled in chunks of 1,024 tokens with contiguous position ids after $\mathbf{x}_b$. Finally, $\mathbf{x}_a$ is prefilled and the answer is decoded from the resulting cache of at most $|\mathbf{x}_b| + nK + |\mathbf{x}_a|$ tokens. The visual tokens resident in memory never exceed about $k+m$ tiles plus one micro-batch, *i.e.* $\mathcal{O}(K)$.

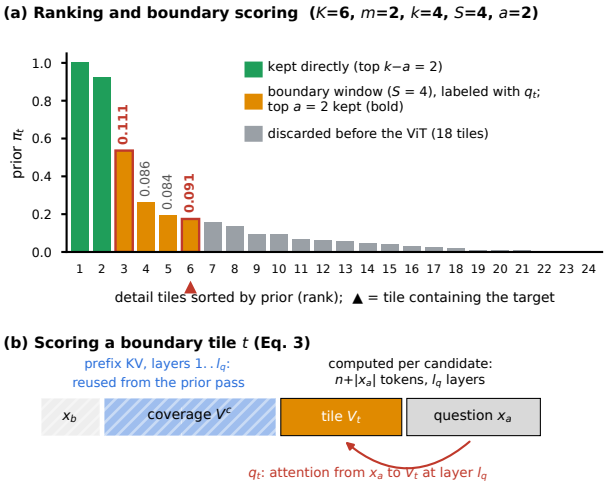


Figure 3. **Boundary scoring** on the example of Fig. 2. (a) Detail tiles sorted by the prior π_t. The top $k - a$ tiles are kept directly, the next S tiles are scored by question attention q_t (Eq. (3)) and the top a are kept, and the rest are discarded before the vision encoder. The target tile is ranked sixth by the prior but has the second-highest q_t in the window. (b) Each candidate reuses the prefix KV cache of $[\mathbf{x}_b; \mathbf{V}^c]$ from the prior pass, so only $n + |\mathbf{x}_a|$ tokens are processed per tile.

Algorithm 1 FoveaTile inference for one image–question pair

Require: image I, text $\mathbf{x}_b, \mathbf{x}_a$, budget K tiles, window S

$m \leftarrow \max(1, \lfloor K/2 \rfloor)$; $a \leftarrow \min(k, \lfloor S/2 \rfloor)$ with $k = K - c_c r_c$
$\mathcal{C} \leftarrow$ KV cache after prefilling $\mathbf{x}_b$
$\mathbf{V}^c \leftarrow \textsc{Encode}(\textsc{CoverageTiles}(I, m))$
π, $\mathcal{C}_{\text{pre}} \leftarrow \textsc{Prior}(\mathbf{x}_b, \mathbf{V}^c, \mathbf{x}_a)$ ▷ Eq. (2), layers $\le l_{\text{hi}}$
$\sigma \leftarrow$ detail tiles of I sorted by π (descending)
$D \leftarrow \sigma[1{:}k-a]$; $W \leftarrow \sigma[k-a+1{:}k-a+S]$ ▷ rest: never encoded
$\mathcal{H} \leftarrow$ empty min-heap of capacity a
for each micro-batch $T \subset D \cup W$ in raster order **do**
  $\{\mathbf{V}_t\}_{t \in T} \leftarrow \textsc{Encode}(T)$
  for $t \in T \cap W$ **do**
    push $(q_t, \mathbf{V}_t)$ into $\mathcal{H}$ ▷ Eq. (3), reuses $\mathcal{C}_{\text{pre}}$
    if $|\mathcal{H}| > a$ **then** pop the lowest-scoring tile from $\mathcal{H}$
$\mathcal{C} \leftarrow \textsc{Prefill}(\mathcal{C},$ tiles $D \cup \mathcal{H}$ in raster order, $\mathbf{V}^c)$
$\mathcal{C} \leftarrow \textsc{Prefill}(\mathcal{C}, \mathbf{x}_a)$; **return** $\textsc{Decode}(\mathcal{C})$

Method	ViT	LLM prefill	KV cache
Uncompressed	N	N	N
VisionZip	N	K	K
FastV, SparseVLM	N	$N \to K$	$\approx K$
IF-Prune	$N^{\ddagger}$	$N \to K$	$\approx K$
FoveaTile (ours)	$K{-}a{+}S$	$K{+}m{+}S^{\dagger}$	K

Table 3. **Cost per image** in units of tiles ($n = 256$ tokens each), with N detail tiles and a budget of K tiles; text tokens are omitted. $N \to K$: all N tiles pass through the layers before pruning, K remain after. ‡Plus a 1B guide VLM on all N tiles. †The m coverage and S boundary tiles pass only through the first l_{hi} and l_q layers. Only FoveaTile is independent of N.

Cost. Tab. 3 summarizes the cost per image. FoveaTile encodes $c_c r_c + (k-a) + S = K - a + S$ tiles. Its language-model work consists of the prior pass over $|\mathbf{x}_b| + nm + |\mathbf{x}_a|$ tokens (l_{hi} layers), S scoring passes of $n + |\mathbf{x}_a|$ tokens (l_q layers), and the main prefill of nK visual tokens. Every term depends on the budget K and the window S, but none depends on the number of tiles N. The KV cache is bounded by the budget. In contrast, every method that prunes after encoding pays $\mathcal{O}(N)$ vision compute, and methods that prune inside the language model also pay $\mathcal{O}(N)$ in the layers before pruning.

4.5. Hyperparameters and calibration

FoveaTile has no trained parameters. The coverage fraction m/K, the window S as a function of the budget, and the default budget are set once on held-out data and frozen for all test sets. The calibration sets are 600 questions sampled from the InfoVQA *train* split and 600 questions from MME-RealWorld excluding its Lite subset and images, so they share no questions or images with any test set. On these sets, $m/K = 0.5$ is within 0.1 points of the best average accuracy and has the most stable worst-task accuracy. With $m = 1$, *i.e.* the thumbnail alone as coverage, accuracy is about 7 points lower on average (Tab. 7). Disabling boundary scoring ($S = 0$) costs up to 10.8 raw points on remote sensing questions, while windows up to four times larger than ours give no reliable gain. We therefore use $S = \{5, 4, 4, 3, 3\}$ for $B = \{1024, 1536, 2048, 2560, 3072\}$. The default budget is $B = 1536$ ($K = 6$, $m = 3$, $S = 4$), where held-out accuracy saturates. The attention layers were fixed in preliminary experiments before this protocol: for InternVL3.5-8B ($L = 36$) we use $\mathcal{L}_p = \{21, \ldots, 28\}$ and $l_q = 20$. The full protocol, including deviations from our pre-registered rules, is given in the supplementary material.

5. Experiments

Our central claim is that the memory and latency of FoveaTile do not grow with the input resolution, while its accuracy matches the strongest token-pruning methods. Sec. 5.2 compares accuracy under memory and token budgets, Sec. 5.3 varies the resolution, and Sec. 5.4 studies the components of FoveaTile.

5.1. Setup

Models and benchmarks. We evaluate on InternVL3.5-4B, -8B, and -14B [21], and use the 8B model unless stated otherwise. We cover three kinds of high-resolution inputs: documents (DocVQA val [12], 5,349 questions; InfoVQA val [13], 2,801), real-world scenes (MME-RealWorld Lite [31], 1,919), and very high-resolution images with small targets (HR-Bench 4K and 8K [20], 800 questions each; V* Bench [24], 191). We report ANLS on DocVQA and InfoVQA, CircularEval accuracy on HR-Bench (a question counts only if all four option orders are answered correctly), and multiple-choice accuracy otherwise. All methods use greedy decoding and the same prompts, and start from the same official dynamic tiling with up to $N_{\max} = 47$ tiles plus a thumbnail.

Baselines. We compare with FastV [3], SparseVLM [30], VisionZip [28], and IF-Prune [18], ported to InternVL3.5 from the official code with the official hyperparameters: FastV and IF-Prune prune at layer 2, IF-Prune uses the released InternVL2.5-1B guide model, SparseVLM prunes at the official relative depths with its pruning rate calibrated per budget, and VisionZip keeps dominant and contextual tokens in each tile. For all methods, the budget B is the number of visual tokens left in the KV cache. Implementation details are in the supplementary material.

Method	Document		Real-world	High resolution			Saving *vs.* uncompressed↑		
	DocVQA	InfoVQA	MME-RW	HR-Bench 4K	V* Bench	Avg. (%)	TTFT	Memory	FLOPs
Upper bound: all visual tokens									
Uncompressed	91.4	78.7	51.6	62.0	83.8	100.0	1.0×	1.0×	1.0×
Budget $B = 1536$ visual tokens (default), ≈21% of all visual tokens									
FastV [3]	84.0 91.9%	61.0 77.4%	46.9 90.9%	<u>59.0</u> 95.2%	79.1 94.4%	90.0 (↓10.0)	<u>2.5×</u>	1.5×	<u>2.9×</u>
SparseVLM [30]	55.1 60.3%	62.2 79.1%	<u>50.2</u> 97.4%	**59.5** 96.0%	<u>79.6</u> 95.0%	85.5 (↓14.5)	2.2×	1.5×	2.4×
VisionZip [28]	48.7 53.3%	47.6 60.4%	41.7 80.9%	49.0 79.0%	70.2 83.8%	71.5 (↓28.5)	**2.7×**	<u>4.2×</u>	**3.2×**
IF-Prune [18]	**88.2** 96.5%	<u>72.7</u> 92.3%	49.3 95.6%	57.5 92.7%	**81.7** 97.5%	<u>94.9</u> (↓5.1)	1.5×	0.7×	1.8×
FoveaTile (ours)	<u>87.0</u>↓ 95.2%	**74.8**↑ 95.0%	**52.1**↑ 100.9%	56.5 91.1%	77.5 92.5%	**95.0** (↓5.0)	2.4×	**5.0×**	**3.2×**
Budget $B = 2560$ visual tokens, ≈35% of all visual tokens									
FastV [3]	<u>89.6</u> 98.0%	70.5 89.5%	50.1 97.2%	<u>59.0</u> 95.2%	<u>81.7</u> 97.5%	95.5 (↓4.5)	<u>2.0×</u>	1.5×	<u>2.2×</u>
SparseVLM [30]	69.8 76.4%	71.1 90.3%	<u>51.8</u> 100.4%	57.0 91.9%	81.2 96.9%	91.2 (↓8.8)	1.7×	1.4×	1.8×
VisionZip [28]	62.8 68.8%	57.9 73.5%	45.9 89.0%	57.5 92.7%	76.4 91.2%	83.0 (↓17.0)	**2.1×**	<u>2.8×</u>	**2.4×**
IF-Prune [18]	<u>89.6</u> 98.0%	<u>74.8</u> 95.0%	49.9 96.8%	**59.5** 96.0%	**83.8** 100.0%	<u>97.1</u> (↓2.9)	1.3×	0.7×	1.5×
FoveaTile (ours)	**90.1** 98.5%	**78.6**↑ 99.8%	**52.2** 101.1%	58.0 93.5%	79.6 95.0%	**97.6** (↓2.4)	1.7×	**3.6×**	<u>2.2×</u>
Budget $B = 3072$ visual tokens, ≈42% of all visual tokens									
FastV [3]	**90.8** 99.4%	73.6 93.5%	49.7 96.3%	**61.0** 98.4%	<u>82.7</u> 98.7%	97.2 (↓2.8)	<u>1.8×</u>	1.5×	<u>2.0×</u>
SparseVLM [30]	69.8 76.3%	72.5 92.1%	<u>51.0</u> 98.8%	57.0 91.9%	**83.8** 100.0%	91.8 (↓8.2)	1.5×	1.3×	1.6×
VisionZip [28]	68.8 75.3%	61.8 78.5%	47.4 91.9%	56.5 91.1%	78.5 93.8%	86.1 (↓13.9)	**1.9×**	<u>2.4×</u>	**2.2×**
IF-Prune [18]	90.1 98.5%	<u>75.1</u> 95.4%	49.8 96.5%	**61.0** 98.4%	82.2 98.1%	<u>97.4</u> (↓2.6)	1.2×	0.7×	1.4×
FoveaTile (ours)	<u>90.3</u> 98.8%	**79.4**↑ 100.8%	**53.5**↑ 103.6%	<u>59.5</u> 96.0%	81.7 97.5%	**99.3** (↓0.7)	1.4×	**2.7×**	1.8×

Table 4. **Accuracy and cost on InternVL3.5-8B.** For each method, the first row is the score on each benchmark and the second its percentage of the uncompressed model; Avg. is the mean percentage. HR-Bench 8K is in Tab. 6. Cost is the saving relative to the uncompressed model (higher is better), averaged over the five benchmarks; absolute values are in the supplementary material. ↑/↓: FoveaTile is significantly better/worse than the strongest baseline on that benchmark (paired bootstrap, 95%); unmarked differences are not significant. Best compressed result per budget in **bold**, second <u>underlined</u>.

Efficiency measurement. All measurements use one NVIDIA A6000 (48 GB), bfloat16, and batch size 1. TTFT is the wall-clock time to the first generated token, including the vision encoder and any scoring or guide model, and we report the median over 50 images per benchmark after one warm-up image. Memory is the peak allocated GPU memory minus the model weights; for IF-Prune it includes the guide model. FLOPs count the vision encoder and the prefill up to the first token.

Protocol. All FoveaTile hyperparameters are fixed on held-out calibration data (Sec. 4.5); no test set is used for any choice. We test differences with a paired bootstrap over questions (or CircularEval groups) at 95% confidence.

5.2. Accuracy under a memory budget

The number of high-resolution requests a GPU can serve is bounded by memory, so we first ask which accuracy each method reaches within a memory limit (Tab. 5). FoveaTile is the most accurate method under every limit. Within 1 GB per image, it retains 99.3% of the uncompressed accuracy, while FastV, SparseVLM, and IF-Prune cannot run at any budget and VisionZip reaches 86.1%. IF-Prune even needs more memory than the uncompressed model (3.0 *vs.* 2.0 GB), mainly for its guide model, so it does not fit within the uncompressed model's own footprint. Without a limit, FoveaTile still has the highest accuracy (99.3% at $B = 3072$).

Tab. 4 compares all methods at equal token budgets. At

Method	Memory limit per image			
	≤0.5 GB	≤1 GB	≤2 GB†	None
FastV	–	–	97.2 (3072)	97.2 (3072)
SparseVLM	–	–	91.8 (3072)	91.8 (3072)
VisionZip	71.5 (1536)	86.1 (3072)	86.1 (3072)	86.1 (3072)
IF-Prune	–	–	–	97.4 (3072)
FoveaTile (ours)	**95.0** (1536)	**99.3** (3072)	**99.3** (3072)	**99.3** (3072)

Table 5. **Accuracy under a memory limit** on InternVL3.5-8B: the best mean percentage of the uncompressed score (five benchmarks of Tab. 4) that each method reaches with budgets $B \in \{1024, \dots, 3072\}$ while its peak memory on top of the model weights stays within the limit (budget in parentheses; –: no budget fits; †the memory of the uncompressed model). FastV, SparseVLM, and IF-Prune need at least 1.3 GB at any budget, and IF-Prune needs more memory than the uncompressed model.

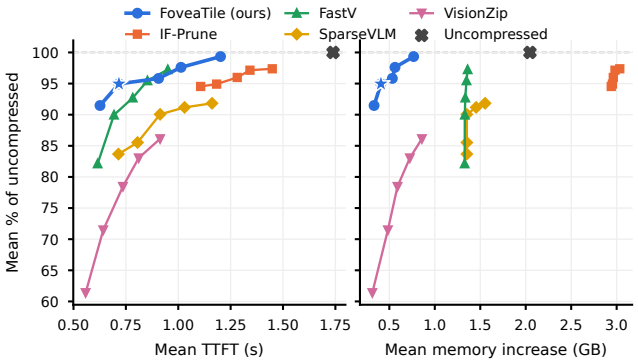


Figure 4. **Accuracy–cost trade-off** on InternVL3.5-8B, averaged over the five benchmarks of Tab. 4, for $B \in \{1024, 1536, 2048, 2560, 3072\}$. The star marks the default FoveaTile budget. FoveaTile uses the least memory at every budget, and every configuration of IF-Prune and SparseVLM is dominated by a FoveaTile configuration that is both faster and more accurate.

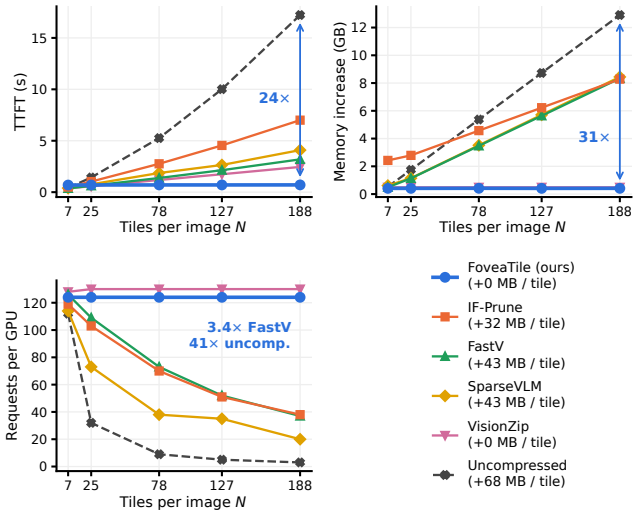


Figure 5. **Cost as the input resolution grows** (HR-Bench 8K, $B = 1536$; the tile cap is raised so that the same images split into 7 to 188 tiles). The legend gives the growth of peak memory per added tile. FoveaTile stays flat in TTFT, memory, and requests per GPU, while every method that prunes inside the language model degrades with N.

the default budget $B = 1536$, which keeps about a fifth of the visual tokens, FoveaTile retains 95.0% of the uncompressed accuracy on average, the highest of all methods, ahead of IF-Prune (94.9%), FastV (90.0%), SparseVLM (85.5%), and VisionZip (71.5%). It needs 5.0× less memory than the uncompressed model, against 1.5× for FastV and SparseVLM and 0.7× for IF-Prune, and it is 1.6× faster than IF-Prune. FoveaTile has the highest average accuracy of all methods at every budget in Tab. 4, reaching 99.3% at $B = 3072$, while using the least memory.

FoveaTile is strongest where the answer depends on the layout of the whole image. On InfoVQA it is significantly better than every baseline at every budget, by 2.1 to 4.3 points over the strongest one, and on MME-RealWorld it even exceeds the uncompressed model (significantly at $B = 3072$, +1.9 points): its high-resolution scenes contain large regions irrelevant to the question, and keeping only the coverage view and the question-relevant tiles removes these distractors, which also explains why more tiles lower the accuracy of the uncompressed model on HR-Bench 8K (Tab. 6). On HR-Bench and V* Bench, which ask about small objects, FoveaTile is within a few points of the best baseline, and none of these differences is significant at any budget.

Fig. 4 shows the full trade-off over five budgets. In memory, FoveaTile is the best curve at every budget: even its largest budget uses less memory than any budget of FastV, SparseVLM, or IF-Prune. In TTFT, FoveaTile reaches the highest accuracy of all methods (99.3% at 1.2 s), and every configuration of IF-Prune and SparseVLM is dominated by a FoveaTile configuration that is both faster and more accurate: at about the same TTFT as IF-Prune at $B = 1536$ (1.2 s), FoveaTile reaches 99.3% instead of 94.9%, and at $B = 1536$ (95.0%) it is more accurate than IF-Prune at any $B \le 1536$ with about 0.6× its TTFT and one seventh of its memory.

Model scale. With the hyperparameters calibrated on the 8B model, FoveaTile reaches 94.5% and 95.2% on InternVL3.5-4B and -14B at $B = 1536$, with peak memory of 0.37 and 0.43 GB, about 7× less than IF-Prune. The full table is in the supplementary material.

5.3. Memory and latency that do not grow with resolution

To isolate the effect of resolution, we raise the tile cap on HR-Bench 8K so that the same images are split into 7 to

	25 tiles			188 tiles		
Method	Acc.	TTFT	Mem.	Acc.	TTFT	Mem.
Uncompressed	56.5	1.44	1.79	49.0	17.22	12.90
FastV	**54.0**	0.63	1.16	42.5	3.19	8.34
SparseVLM	53.5	0.76	1.18	51.5	4.09	8.44
VisionZip	44.5	**0.58**	0.49	27.0	2.45	0.49
IF-Prune	52.5	1.04	2.81	**52.0**	7.00	8.28
FoveaTile (ours)	52.0	0.70	**0.42**	47.0	**0.71**	**0.41**

Table 6. **Higher input resolution at a fixed budget** ($B = 1536$) on HR-Bench 8K. Raising the tile cap from $N_{\max} = 47$ to 200 splits each image into 188 tiles instead of about 25. Acc. is CircularEval accuracy, TTFT in seconds, Mem. in GB.

Variant	Info	MME	Local	Avg.	Worst	TTFT
FoveaTile	94.4	97.1	100.1	95.8	85.4	0.92
Coverage only, $m{=}K$	95.8	95.6	95.8	95.7	77.5	0.58
Thumbnail, $m{=}1$	86.6	90.8	95.6	88.7	75.7	–
Prior only, $S{=}0$	94.5	93.6	92.6	94.1	78.5	0.73
Window 2×	94.2	97.6	98.4	95.9	85.0	1.11
Window 4×	94.5	96.6	97.0	95.5	80.9	1.48
Random prior	85.9	87.7	90.2	86.8	70.6	–

Table 7. **Ablations** on the held-out calibration sets of Sec. 4.5 (Info, MME), as a percentage of the uncompressed score. Local: MME-RealWorld remote sensing, monitoring, and autonomous driving. Worst: lowest percentage among InfoVQA and the five MME-RealWorld subtasks. Window sizes are relative to the default S; TTFT in seconds.

188 tiles, and measure cost at $B = 1536$ (Fig. 5). The peak memory of FoveaTile does not grow at all: it stays at 0.41 GB from 7 to 188 tiles, because no stage of FoveaTile ever holds more than the budget. Every method that prunes inside the language model grows linearly with N, by 32 to 43 MB per added tile (68 MB for the uncompressed model), and needs over 8 GB at 188 tiles, 20× more than FoveaTile. TTFT behaves the same way: FoveaTile stays at 0.7 s, whereas FastV and IF-Prune become 4.5× and 10× slower than FoveaTile at 188 tiles, and the uncompressed model 24× slower. VisionZip also keeps its memory flat, since it prunes before the language model, but it still encodes every tile, so its TTFT grows with N, and its accuracy is 23 points lower on average (Tab. 4). FoveaTile is the only method whose memory and latency are both independent of the resolution. Beyond about 35 tiles, FoveaTile is the fastest of all methods, and its lead grows with N.

Memory determines how many requests a GPU can serve. We measure this directly: on one A6000 (33 GB free after the 8B weights), we prefill requests one after another, keep the KV cache of each, as if it were still decoding, and count the requests that complete before the GPU runs out of memory. FoveaTile holds 124 requests at every resolution, from 7 to 188 tiles. With FastV and IF-Prune this number falls from 103–109 at 25 tiles to 37–38 at 188 tiles, with SparseVLM to 20, and with the uncompressed model from 32 to 3. At 188 tiles, FoveaTile serves 3.4× more requests than FastV, 3.3× more than IF-Prune, 6.2× more than SparseVLM, and 41× more than the uncompressed model.

Tab. 6 reports accuracy at both resolutions. Splitting the image more finely at a fixed budget lowers accuracy for every method, and even for the uncompressed model (56.5 to 49.0). Among the compressed methods, FoveaTile has the best accuracy per unit cost: at 188 tiles it is 10× faster than IF-Prune and 6× faster than SparseVLM with 20× less memory, and it is more accurate than FastV (47.0 *vs.* 42.5) at 4.5× less TTFT. With a larger budget ($B = 2560$), FoveaTile matches the best compressed methods at 188 tiles (52.5) and exceeds the uncompressed model (49.0), which suffers from the very long visual sequence.

5.4. Ablations

Tab. 7 ablates the components of FoveaTile on the held-out calibration sets, averaged over five budgets. Using the thumbnail alone as coverage ($m = 1$) loses 7.1 points on average, so giving the coverage tiles half of the budget is essential. Using only coverage tiles ($m = K$) gives the same average as FoveaTile at lower TTFT, but its worst task drops from 85.4% to 77.5% and the local-detail subtasks from 100.1% to 95.8%: detail tiles do not raise the average, they prevent failures on questions about small regions. Removing boundary scoring ($S = 0$) costs 1.7 points on average, 6.9 on the worst task, and 7.5 on local-detail questions, so ranking by the prior alone is not enough. Larger windows add TTFT without improving accuracy. Finally, replacing the question-aware prior with a random ranking, while keeping boundary scoring, costs 9.0 points on average and 14.8 on the worst task: which detail tiles reach the window matters most, and the attention of the VLM over the coverage tiles is what finds them. These accuracy results complement the recall analysis of Sec. 3.2.

6. Conclusion

We presented FoveaTile, a training-free method that bounds the memory of high-resolution VLMs by deciding which tiles to encode before the vision encoder, since token pruning removes tokens only after they have been computed. On InternVL3.5, FoveaTile keeps 95.0% of the uncompressed accuracy with 5.0× less memory and is the most accurate method under every memory limit. Its peak memory stays constant from 7 to 188 tiles, so one GPU serves 41× more requests than the uncompressed model, and its behavior carries over from 4B to 14B parameters without re-tuning.

Because every detail tile is chosen from a low-resolution view, the recall of this prior for very small objects is the main lever for further gains. Improving this recall, and ex-

tending the selection to native-resolution encoders and to video, are directions for future work.

LLM Usage Statement

We used Claude Code (Anthropic) to help write and run experiment code, analyze results, generate tables and figures, and draft and edit the text of this paper, under the direction of the authors. The authors reviewed these outputs and take responsibility for all content.

References

[1] Kazi Hasan Ibn Arif, JinYi Yoon, Dimitrios S. Nikolopoulos, Hans Vandierendonck, Deepu John, and Bo Ji. HiRED: Attention-guided token dropping for efficient inference of high-resolution vision-language models. In *AAAI*, 2025. 2

[2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *ICLR*, 2023. 2

[3] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *ECCV*, 2024. 1, 2, 6, 7, S3

[4] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. 1, 2

[5] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *MLSys*, 2024. 5

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023. 1, 2

[7] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, et al. LLaVA-OneVision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2

[8] Geng Li, Jinglin Xu, Yunzhen Zhao, and Yuxin Peng. DyFo: A training-free dynamic focus visual search for enhancing LMMs in fine-grained visual understanding. In *CVPR*, 2025. 2, 4

[9] Zhang Li, Biao Yang, Qiang Liu, Zhiyin Ma, Shuo Zhang, Jingxu Yang, Yabo Sun, Yuliang Liu, and Xiang Bai. Monkey: Image resolution and text label are important things for large multi-modal models. In *CVPR*, 2024. 2

[10] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. LLaVA-NeXT: Improved reasoning, OCR, and world knowledge. https://llava-vl.github.io/blog/2024-01-30-llava-next/, 2024. 1, 2

[11] Xuyang Liu, Ziming Wang, Junjie Chen, Yuhang Han, Yingyao Wang, Jiale Yuan, Jun Song, Siteng Huang, and Honggang Chen. Global compression commander: Plug-and-play inference acceleration for high-resolution large vision-language models. In *AAAI*, 2026. 2

[12] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A dataset for VQA on document images. In *WACV*, 2021. 6

[13] Minesh Mathew, Viraj Bagal, Rubèn Pérez Tito, Dimosthenis Karatzas, Ernest Valveny, and C. V. Jawahar. InfographicVQA. In *WACV*, 2022. 6

[14] Volodymyr Mnih, Nicolas Heess, Alex Graves, and Koray Kavukcuoglu. Recurrent models of visual attention. In *NeurIPS*, 2014. 1

[15] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. 4

[16] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024. 2

[17] Haozhan Shen, Kangjia Zhao, Tiancheng Zhao, Ruochen Xu, Zilun Zhang, Mingwei Zhu, and Jianwei Yin. ZoomEye: Enhancing multimodal LLMs with human-like zooming capabilities through tree-based image exploration. In *EMNLP*, 2025. 2, 4

[18] Guohao Sun, Yufei Wang, Sizhuo Ma, Yuege Xie, Yuting Cheng, Zhiqiang Tao, and Jian Wang. IF-Prune: Information-flow guided token pruning for efficient vision-language models. In *CVPR*, 2026. 1, 2, 6, 7, S4

[19] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 2

[20] Wenbin Wang, Liang Ding, Minyan Zeng, Xiabin Zhou, Li Shen, Yong Luo, and Dacheng Tao. Divide, conquer and combine: A training-free framework for high-resolution image perception in multimodal large language models. In *AAAI*, 2025. 6

[21] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, et al. InternVL3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025. 1, 2, 6

[22] Wenbin Wang, Yongcheng Jing, Liang Ding, Yingjie Wang, Li Shen, Yong Luo, Bo Du, and Dacheng Tao. Retrieval-augmented perception: High-resolution image perception meets visual RAG. In *ICML*, 2025. 2, 4

[23] Yulin Wang, Kangchen Lv, Rui Huang, Shiji Song, Le Yang, and Gao Huang. Glance and focus: A dynamic approach to reducing spatial redundancy in image classification. In *NeurIPS*, 2020. 1

[24] Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal LLMs. In *CVPR*, 2024. 1, 2, 3, 4, 5, 6

[25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *ICLR*, 2024. 1, 2

[26] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, et al. PyramidDrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. In *CVPR*, 2025. 2

[27] Ruyi Xu, Yuan Yao, Zonghao Guo, Junbo Cui, Zanlin Ni, Chunjiang Ge, et al. LLaVA-UHD: An LMM perceiving any aspect ratio and high-resolution images. In *ECCV*, 2024. 2

[28] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *CVPR*, 2025. 1, 2, 6, 7, S4

[29] Jiarui Zhang, Mahyar Khayatkhoei, Prateek Chhikara, and Filip Ilievski. MLLMs know where to look: Training-free perception of small visual details with multimodal LLMs. In *ICLR*, 2025. 2, 4

[30] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, et al. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *ICML*, 2025. 1, 2, 6, 7, S3

[31] Yi-Fan Zhang, Huanyu Zhang, Haochen Tian, Chaoyou Fu, Shuangqing Zhang, Junfei Wu, et al. MME-RealWorld: Could your multimodal LLM challenge high-resolution real-world scenarios that are difficult for humans? In *ICLR*, 2025. 6

[32] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. 1, 2

[33] Zhi Zhang, Srishti Yadav, Fengze Han, and Ekaterina Shutova. Cross-modal information flow in multimodal large language models. In *CVPR*, 2025. 4

Look Before You Encode: Foveated Tile Streaming for Resolution-Independent VLM Inference

Supplementary Material

	HR-4K		HR-8K		V* Bench			
Method	S	C	S	C	Attr.	Pos.	Hit	Miss
Uncompressed	77.0	47.0	66.0	47.0	85.2	81.6	86.8	76.4
FastV	73.0	**45.0**	63.0	**45.0**	80.0	77.6	83.1	**69.1**
SparseVLM	75.0	44.0	66.0	41.0	80.0	**78.9**	85.3	65.5
VisionZip	62.0	36.0	59.0	30.0	75.7	61.8	73.5	61.8
IF-Prune	**76.0**	39.0	**68.0**	37.0	**85.2**	76.3	86.8	**69.1**
FoveaTile (ours)	71.0	42.0	61.0	43.0	77.4	77.6	**88.2**	50.9

Table S1. **Where FoveaTile helps and where it fails** ($B = 1536$). HR-Bench is split into single- (S) and cross-instance (C) questions (CircularEval), V* Bench into attribute and relative-position questions. Hit / Miss split V* Bench by whether the detail tiles selected by FoveaTile cover the target (136 hit, 55 miss); every method is evaluated on the same split.

S1. Where FoveaTile helps and where it fails

Tab. S1 breaks down the high-resolution benchmarks by question type. On questions that involve several objects or the global arrangement of the scene (HR-Bench cross-instance, V* Bench relative position), FoveaTile is on par with or better than the baselines, *e.g.* 43.0 *vs.* 37.0 for IF-Prune on HR-Bench 8K cross-instance. Its gap is concentrated on questions about a single small object (HR-Bench single-instance, V* Bench attributes).

V* Bench provides target boxes, which tell us why. When the selected detail tiles cover the target (136 of 191 questions), FoveaTile answers 88.2% correctly, on par with the best baseline and the uncompressed model (86.8%), since the target is seen at native resolution without the distraction of the other tiles. When the target is missed, accuracy drops to 50.9%. The misses come from the prior, not from boundary scoring: most missed targets are ranked outside the scoring window, while targets that enter the window are selected 72–94% of the time, depending on the budget (Tab. 2). The prior is computed on the coverage tiles, in which more than half of the V* targets are smaller than one ViT patch (14 px), so a small budget can leave them out of the window. Methods that encode every tile do not have this failure mode, but pay $\mathcal{O}(N)$ for it.

S2. Details of the reliability analysis

Tab. S2 extends Tab. 1 to all budgets with saved selections. Boundary scoring only reorders the tiles inside the window, so the two values coincide at every K outside the window. The coverage view, and thus the prior, changes with the budget because $m = \lfloor K/2 \rfloor$. Hits follow the definition of Sec. 3.2. In this table, the top of the refined ranking consists of the directly kept tiles in prior order, followed by the window in the order of q_t and the remaining tiles in prior order.

Why the top of the ranking needs a second stage. At $B = 1536$, the first-ranked tile of the prior is the same as with the neutral question "Describe this image." on 90.6% of the questions, and the same as with the swapped question on 91.6%; the median correlation between the prior with the real and the neutral question is 0.93. This tile falls into the top quarter of the image on 63.4% of the questions (62.3%, 82.7%, and 82.2% at $B = 1024$, 2048, and 3072), whereas the top quarter holds the center of a target on only 4.7% of the questions. The dependence on the question therefore shows below the first rank, which is where the prior separates from the swapped-question prior in Fig. 1. We have not established the cause of this pattern, and we did not change the method in response to it.

Upper bound of the scorer. If q_t is computed for all detail tiles, its first-ranked tile covers the target on 56.5% of the questions, and its top k tiles on 84.8%, against 71.2% for the selection of FoveaTile at $B = 1536$. This requires encoding every tile and thus gives up the bound on compute, so it serves only as a reference. The neutral question reaches a Recall@5 of 22.0%, close to a center-first ranking (22.5%), and the swapped question 49.2%.

Target size. Measured in the coverage tiles at $B = 1536$, 63.4% of the questions have a target whose short side is below one ViT patch (14 px). Boundary scoring raises the hit rate from 59.5% to 66.9% on these questions, and from 77.1% to 78.6% on the others.

S3. Model scale

Tab. S3 repeats the default setting on InternVL3.5-4B and -14B, using the hyperparameters calibrated on the 8B model; only the attention layers are mapped by relative depth. The results carry over without re-tuning. On all three models, FoveaTile is among the two most accurate methods (94.5%, 95.0%, and 95.2%), well ahead of FastV (89.4 to 91.4), SparseVLM (84.5 to 85.5), and VisionZip (59.0 to 76.8). The memory advantage holds at every scale: FoveaTile uses the least memory on all three models (0.37,

	$B = 1024$		$B = 1536$		$B = 2048$		$B = 3072$	
K	Prior	Scored	Prior	Scored	Prior	Scored	Prior	Scored
1	3.1	51.3	3.1	3.1	3.1	3.1	3.1	3.1
2	27.7	69.1	26.7	30.4	23.6	23.6	27.7	27.7
3	51.3	71.2	50.3	61.3	46.1	61.8	42.9	42.9
4	68.6	72.8	67.0	71.2	59.7	72.8	58.1	58.1
5	73.8	73.8	72.8	74.3	69.1	77.0	68.1	68.1
6	78.5	78.5	78.5	78.5	77.5	78.0	77.5	80.1
8	85.3	85.3	85.9	85.9	84.3	84.3	85.3	85.3
12	95.3	95.3	94.8	94.8	92.1	92.1	93.2	93.2

Table S2. **Recall@K (%) of the coarse prior and after boundary scoring** on V* Bench, at every budget with saved selections.

Method	DocVQA	InfoVQA	MME-RW	HR-Bench 4K	V* Bench	Avg. (%)	TTFT (s)↓	Mem. (GB)↓
InternVL3.5-4B, $B = 1536$								
Uncompressed (score)	91.0	77.6	49.5	59.5	81.2	100.0	1.24	1.81
FastV	89.9	77.8	93.4	89.9	96.1	89.4	0.56	1.03
SparseVLM	62.1	79.2	94.8	89.1	97.4	84.5	0.63	1.07
VisionZip	51.3	49.8	65.9	52.9	74.8	59.0	0.53	0.40
IF-Prune	**97.1**	91.8	95.0	90.8	**98.1**	**94.6**	1.05	2.70
FoveaTile (ours)	95.1	**94.7**	**95.5**	**92.4**	94.8	94.5	**0.50**	**0.37**
InternVL3.5-8B, $B = 1536$								
Uncompressed (score)	91.4	78.7	51.6	62.0	83.8	100.0	1.74	2.05
FastV	91.9	77.4	90.9	95.2	94.4	90.0	0.69	1.33
SparseVLM	60.3	79.1	97.4	**96.0**	95.0	85.5	0.81	1.35
VisionZip	53.3	60.4	80.9	79.0	83.8	71.5	**0.64**	0.48
IF-Prune	**96.5**	92.3	95.6	92.7	**97.5**	94.9	1.18	2.95
FoveaTile (ours)	95.2	**95.0**	**100.9**	91.1	92.5	**95.0**	0.72	**0.41**
InternVL3.5-14B, $B = 1536$								
Uncompressed (score)	92.8	80.2	52.0	61.0	84.3	100.0	2.76	2.48
FastV	91.4	79.6	91.2	96.7	**98.1**	91.4	0.95	1.73
SparseVLM	64.9	76.7	**97.5**	92.6	93.8	85.1	1.14	1.76
VisionZip	59.9	62.7	84.4	86.9	90.1	76.8	**0.86**	0.55
IF-Prune	**96.9**	92.7	96.6	95.1	96.9	**95.6**	1.45	3.27
FoveaTile (ours)	96.4	**94.5**	96.7	**97.5**	90.7	95.2	1.11	**0.43**

Table S3. **Model scale.** Percentage of the uncompressed score at $B = 1536$ on InternVL3.5-4B, -8B, and -14B (the uncompressed rows give raw scores). FoveaTile uses the hyperparameters calibrated on the 8B model without re-tuning; its attention layers are mapped by relative depth. Best compressed result per model in bold.

0.41, and 0.43 GB), since its footprint is set by the budget and not by the input, whereas IF-Prune needs 2.70 to 3.27 GB, about 7× more, and FastV and SparseVLM over 1 GB. FoveaTile is also faster than IF-Prune on every model (2.1× on 4B, 1.3× on 14B), and on 4B it is the fastest of all methods (0.50 s). On InfoVQA, FoveaTile remains the best method on every model, and on HR-Bench 4K it is the best on 4B and 14B.

S4. Calibration protocol

All hyperparameters of FoveaTile were set on two held-out calibration sets. The test sets were used only to report results and to diagnose errors; any change suggested by a diagnosis had to pass on the calibration sets first.

Calibration sets. *InfoVQA train* consists of 600 questions sampled with a fixed seed from the train split (ANLS). *MME-RW calib* is drawn from the full English MME-RealWorld (23,609 questions): we remove the questions of the Lite subset (1,787) and all other questions on its images (2,654), and sample 120 of the remaining 19,168 questions from each of the five subtasks (accuracy, 600 questions). The result shares no questions and no images with the Lite test set. DocVQA has no train split with answers, so InfoVQA train also represents documents.

Metrics. Retention is the score divided by the uncompressed score on the same set (83.9 and 59.5). We report its average over the two sets and its worst value over six groups (InfoVQA and the five subtasks). TTFT is the median over 50 images after one warm-up image, with one process per GPU. Comparisons use a paired bootstrap on the same ques-

m	1024	1536	2048	2560	3072	Mean
1	57.9	61.2	63.9	65.2	68.4	63.3
$K/2$	63.9	68.4	68.8	70.7	71.0	**68.5**
$3K/4$	65.2	67.6	68.3	70.0	70.9	68.4
K	63.5	68.8	69.1	70.1	71.8	68.6

Table S4. **Stage A: coverage fraction.** Score averaged over the two calibration sets, per budget B.

	$m = K/2$			$m = 3K/4$			$m = K$		
B	Avg.	Min	TTFT	Avg.	Min	TTFT	Avg.	Min	TTFT
1024	89.7	76	0.90	91.2	77	0.69	88.5	80	0.40
1536	95.7	85	0.99	94.3	72	0.89	96.2	86	0.45
2048	95.9	85	1.08	95.5	72	1.00	96.0	74	0.56
2560	98.9	94	1.19	97.9	82	1.15	97.7	69	0.69
3072	99.3	86	1.41	99.1	83	1.32	100.1	78	0.80

Table S5. **Retention (%) and TTFT (s)** for the three plausible coverage fractions, before the window was calibrated. Avg.: average over the two sets. Min: worst of the six groups.

tions with 2,000 resamples (4,000 in the candidate tests) at 95% confidence.

Stage A: coverage fraction and default budget. We swept $m \in \{1, K/4, K/2, 3K/4, K\}$ with a provisional window. Using only the thumbnail ($m = 1$) loses about 5 points at every budget (Tab. S4), while m/K from 0.5 to 1 forms a plateau whose differences are mostly not significant. Since the averages tie, we chose by stability: $m = K/2$ has the most stable worst-group retention (Tab. S5). We set $B = 1536$ as the default because retention saturates there (90% at $B = 1024$, 96% at $B = 1536$, no further gain at $B = 2048$). Pure coverage ($m = K$) is about twice as fast and is reported as an ablation.

Stage B: scoring window. We fixed $m = \lfloor K/2 \rfloor$ and swept $S \in \{0, S_0/2, S_0, 1.5S_0, 2S_0\}$, where S_0 is the provisional window (Tab. S6). Removing the window ($S = 0$) is significantly worse on local-detail questions, *e.g.* -10.8 points [-18, -4] on Remote Sensing at $B = 1024$ and 1536. Larger windows bring no reliable gain and add TTFT.

Attention layers. The layers $\mathcal{L}_p = \{21, \ldots, 28\}$ and $l_q = 20$ were chosen on InfoVQA val before this protocol existed, and InfoVQA val is also a test set. They were not re-selected on the calibration sets. For the 4B and 14B models they are mapped by relative depth, by a rule fixed before those models were run.

Rejected candidates. Diagnosing errors on the test sets suggested the four changes in Tab. S7. Before each test we fixed three criteria: average retention at most 0.5 points below v1.0 at every budget, a significant gain on the local-detail questions of MME-RW (Remote Sensing, Monitoring, Autonomous Driving; 360 questions), and at most 0.3 s of added TTFT at $B = 1536$. None of the candidates passed its criteria, so we kept the configuration as frozen.

Deviations from the pre-registered rules.

1. The rule for Stage B selected $S = \{5, 13, 4, 3, 14\}$: the smallest S within one point of the best average and worst-group retention. A worst group has only 120 questions, so that the rule was driven by noise, and the differences to $S_0/2$ are not significant. We therefore used $S_0/2$ at every budget.
2. The sweep of the growing window added $f = 0.2$, since larger f is very costly on MME-RW. The criteria did not change.
3. Sorted options were added after the results of the other query variants were known. The criteria did not change, and the candidate was rejected.
4. An earlier configuration had its window table and coverage fraction tuned on test data. We discarded it and recalibrated as described here.

S5. Baseline implementation

We ported each baseline to InternVL3.5 from its official code and kept the selection algorithm and official hyperparameters. All methods share the prompts, the tiling, the greedy decoding, and the measurement code. Budget B always means the number of visual tokens left in the KV cache, so a method keeps $\min(B, \text{all tokens})$. The points below are the only decisions that the official code does not determine.

FastV [3]. We follow the official in-place variant: at layer 2, the attention of the last token ranks the visual tokens, the top B are kept, and the sequence is actually shortened, with the original position ids retained. The official in-place code does not support a KV cache, so we added multi-token decoding with per-layer cache lengths. The pruning decision is unchanged.

SparseVLM [30]. We implement the rank-based removal of the paper at the official relative depths (layers 2, 6, and 15 of 32, scaled to the depth of the model). The paper publishes no value of λ, so for each benchmark and budget we search the smallest λ for which the three stages alone reach between B and $1.5B$ tokens, and use the median over 32 images. This step uses only token counts, not answers; images that need no compression are excluded. Remaining excess tokens are clipped at the final stage. An image with at most B tokens is not pruned ($\lambda = 0$), as for all other

B	S	Avg. retention (%)	Worst group (%)	TTFT (s)
1024	0 / **5** / 10 / 15 / 20	85.9 / **89.6** / 89.7 / 88.6 / 89.3	61.5 / **77.7** / 75.5 / 73.8 / 73.8	0.43 / **0.67** / 0.89 / 1.12 / 1.35
1536	0 / **4** / 9 / 13 / 18	93.2 / **95.6** / 95.7 / 95.5 / 94.5	80.0 / **85.1** / 84.6 / 89.2 / 78.5	0.58 / **0.77** / 0.99 / 1.18 / 1.41
2048	0 / **4** / 8 / 12 / 16	95.1 / **96.7** / 95.9 / 95.5 / 96.2	75.4 / **84.6** / 84.6 / 81.5 / 83.1	0.72 / **0.92** / 1.08 / 1.26 / 1.45
2560	0 / **3** / 7 / 10 / 14	97.7 / **98.8** / 98.9 / 97.5 / 97.5	89.2 / **93.6** / 93.8 / 83.1 / 80.0	0.86 / **1.02** / 1.19 / 1.30 / 1.48
3072	0 / **3** / 7 / 10 / 14	98.5 / **98.2** / 99.3 / 98.3 / 100.3	86.2 / **86.2** / 86.2 / 86.2 / 89.2	1.05 / **1.23** / 1.41 / 1.53 / 1.72

Table S6. **Stage B: scoring window** with $m = \lfloor K/2 \rfloor$. The adopted value of S is in bold.

Candidate	Result on the calibration sets
Window grows with the grid	No $f \in \{0.2, \ldots, 0.5\}$ passes all criteria; local-detail Δ at $f = 0.2$: +0.67 [−0.44, +1.78]
Question-stem query, top-patch aggregation	Worse on local detail for all eight variants, *e.g.* −3.33 for the stem query
Options sorted before scoring	Local detail −0.19 [−0.93, +0.46]; retention −0.56 at $B = 1536$, misses the limit of −0.5
Attention-head selection, question-free correction	Recall +4.2 on a synthetic set, but retention −1.60, −0.83, −0.85; local detail −2.22 [−4.35, −0.09]

Table S7. **Rejected candidates.** All were tested with the criteria below, and each decision used only the calibration sets.

methods; the official experiments use a fixed input that always exceeds the budget, so the official code has no such case.

VisionZip [28]. Dominant and contextual tokens are computed per tile in the feature layer of the vision encoder. The contextual share of 5/32 reproduces all pairs in the tables of the paper, and we apply it to other sizes as well. The official API takes the numbers per crop, so we split B evenly over the tiles. InternVL merges 2×2 patches by pixel shuffle before the projector, so tokens are selected and merged after pixel shuffle and before the projector, averaging scores and keys in each group as in the official Qwen2.5-VL code. The CLS token serves only as the query, since InternVL drops it.

IF-Prune [18]. We load the released guide model (InternVL2.5-1B with its LoRA and importance head) without retraining. It runs once on all tiles and the question, as in the official evaluation code, and the main model keeps the top B tokens at layer 2, which corresponds to a keep ratio of B divided by the number of tokens. Prefill and decoding follow our FastV path. The official experiments use InternVL2 main models, so applying the guide to InternVL3.5 is an extrapolation of their cross-model result. Both models produce 256 tokens per tile in the same order. Memory and TTFT include the guide model.